

第二次实验：C++ 标准线程生产者-消费者模型

2023302121009 刘鼎琨

一、实验源码

1. BufferPush 代码:

```
45 bool BufferPush(BoundedBuffer& buffer, int item) {  
46     unique_lock<mutex> lock(buffer.mtx);  
47  
48     buffer.not_full.wait(lock, [&buffer] {  
49         return buffer.count < static_cast<int>(buffer.data.size());  
50     });  
51  
52     buffer.data[buffer.tail] = item;  
53  
54     buffer.tail = (buffer.tail + 1) % buffer.data.size();  
55     buffer.count++;  
56  
57     lock.unlock();  
58  
59     buffer.not_empty.notify_one();  
60  
61     return true;  
62 }
```

图 1 BufferPush 代码

2. BufferPop 代码:

```
64 bool BufferPop(BoundedBuffer& buffer, int& item) {  
65     unique_lock<mutex> lock(buffer.mtx);  
66  
67     buffer.not_empty.wait(lock, [&buffer] {  
68         return buffer.count > 0;  
69     });  
70  
71     item = buffer.data[buffer.head];  
72  
73     buffer.head = (buffer.head + 1) % buffer.data.size();  
74     buffer.count--;  
75  
76     lock.unlock();  
77  
78     buffer.not_full.notify_one();  
79  
80     return true;  
81 }
```

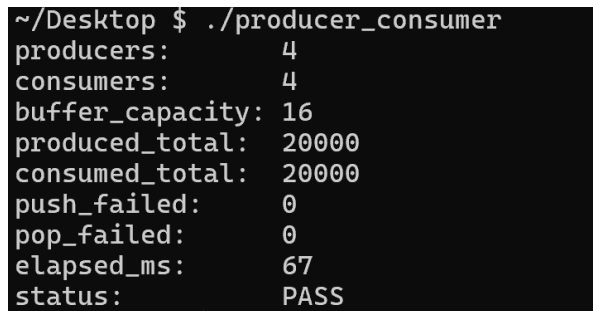
图 2 BufferPop 代码

二、运行结果

1. 执行终端指令：

```
D:\并行与分布式计算\课程实验> g++ -std=c++11  
producer_consumer_cpp_starter.cpp -o producer_consumer.exe  
D:\并行与分布式计算\课程实验> ./producer_consumer
```

2. 运行结果：



```
~/Desktop $ ./producer_consumer  
producers:      4  
consumers:      4  
buffer_capacity: 16  
produced_total: 20000  
consumed_total: 20000  
push_failed:    0  
pop_failed:     0  
elapsed_ms:     67  
status:         PASS
```

图 3 运行结果截图

三、实验分析

1. 为什么生产者要等待 `not_full`?

缓冲区是有大小限制的（本实验中是 16）。如果生产者生产速度远快于消费者，且不检查是否已满就盲目写入，就会覆盖掉还没有被消费的旧数据，导致数据丢失或内存破坏。

2. 为什么消费者要等待 `not_empty`?

如果缓冲区是空的，消费者若强行读取，读到的将是初始化时的 0 或者内存里的垃圾值。如果不使用等待机制，消费者就需要用一个 `while(true)` 循环不断检查是否有数据。这会白白占用 CPU 资源，却什么都没做。

3. `mutex` 和 `condition_variable` 分别解决什么问题？

`Mutex` 是安全层面的考虑，他解决“竞态条件”，能确保同一时间只有一个线程能操作缓冲区索引和计数器。

`Condition_variable` 是同步层面的考虑，他解决“顺序和时机”。让线程在特定条件不满足时休眠，在满足时被唤醒。

4. 为什么有几个消费者，就要放几个 STOP_ITEM?

这是为了确保每一个消费者线程都能安全退出。如果只放 1 个 STOP_ITEM，那么只有 1 个消费者会退出，另外 3 个消费者会永远阻塞在 `not_empty.wait()` 上，导致主线程在 `t.join()` 时永久卡死。

四、项目逻辑

